AI Practicals 1–8: Exam Cheat Sheet

Built from your practical journal. Idea: **don't memorise 20 programs, memorise 5 templates + the small changes between programs.**

0. The 5 templates (learn these, everything else is a tweak)

#	Template	Used in
T1	Table-driven agent (dict lookup)	P1 Q1
T2	Search skeleton (frontier + visited + neighbors)	P1 Q2, P2, P3, P4
T3	Bayes formula (plain Python)	P5 Q1
T4	ML workflow: load → split → fit → predict → score	P5 Q2, P6, P7, P8
T5	pgmpy: network → CPDs → VariableElimination	P5 Q3

---

1. T1: Table-driven agent (P1 Q1)

```
table = {
    ('A', 'Dirty'): 'Suck',
    ('A', 'Clean'): 'Right',
    ('B', 'Dirty'): 'Suck',
    ('B', 'Clean'): 'Left'
}

def table_driven_agent(percept):
    return table.get(percept, "No Action")

for p in [('A','Dirty'), ('B','Clean'), ('C','Dirty')]:
    print("Percept:", p, "Action:", table_driven_agent(p))
```

Remember: key = (location, status), value = action, `.get(key, default)` handles unknown percepts.

PEAS (write this in theory part)

	Vacuum cleaner	Autonomous taxi
Performance	Cleanliness, less time/energy	Safety, speed, comfort, legal, profit
Environment	Rooms A/B, dirt	Roads, traffic, pedestrians, weather
Actuators	Wheels, suction	Steering, accelerator, brake, horn, signals
Sensors	Dirt sensor, location sensor	Camera, GPS, lidar, speedometer

Environment classification

Property	Vacuum (2-room)	Taxi
Observable	Partially (sees only its own square)	Partially
Deterministic?	Deterministic	Stochastic
Episodic/Sequential	Episodic	Sequential

---

2. T2: Search skeleton (the most important template)

All four algorithms (BFS, UCS, Greedy, A*) are the **same loop**. Only 3 things change:

Algorithm	Frontier type	Ordered by	Goal test	Optimal?
BFS	`deque` (FIFO) `popleft()`	order of arrival	when generated (or popped)	Yes (equal step cost)
DFS	stack / recursion (LIFO)	last in first out	when popped	No
UCS	`heapq`	**g(n)** = path cost so far	when **popped**	Yes
Greedy BFS	`heapq`	**h(n)** only	when popped	No
A*	`heapq`	**f(n) = g(n) + h(n)**	when popped	Yes (if h admissible)

Memory line: `UCS = g`, `Greedy = h`, `A* = g + h`. The push line is the only difference:

```
heapq.heappush(pq, (cost + dist, neighbor, path + [neighbor]))        # UCS
(g)
heapq.heappush(pq, (manhattan(neighbor), neighbor, path + [neighbor])) #
Greedy (h)
heapq.heappush(pq, (new_g + new_h, new_g, neighbor, path + [neighbor])) # A*
(f, g)
```

2a. 8-puzzle helpers (used in P1 Q2, P3, P4)

Two styles appear in your journal. Pick ONE and stick to it in the exam.

Style 1: flat tuple (BFS in P1). State = `(1,2,3,4,0,6,7,5,8)`, 0 is blank.

```
from collections import deque

moves = {'Up': -3, 'Down': 3, 'Left': -1, 'Right': 1}

def get_neighbors(state):
    neighbors = []
    zero = state.index(0)
    for move, d in moves.items():
        new = zero + d
        if move == 'Left'  and zero % 3 == 0: continue   # left edge
        if move == 'Right' and zero % 3 == 2: continue   # right edge
        if new < 0 or new >= 9: continue                  # top/bottom
        s = list(state)
        s[zero], s[new] = s[new], s[zero]
        neighbors.append((tuple(s), move))
    return neighbors
```

*Style 2: 2D list (P3 Greedy and P4 A).** Needs `state_to_tuple` because lists can't go into a `set` / `dict`.

```
def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0: return (i, j)

def get_neighbors(state):
    neighbors = []
    x, y = find_blank(state)
    for dx, dy in [(-1,0), (1,0), (0,-1), (0,1)]:
        nx, ny = x+dx, y+dy
        if 0 <= nx < 3 and 0 <= ny < 3:
            s = [row[:] for row in state]          # copy!
            s[x][y], s[nx][ny] = s[nx][ny], s[x][y]
            neighbors.append(s)
    return neighbors

def state_to_tuple(state):
    return tuple(tuple(r) for r in state)
```

Manhattan distance (P3 and P4, identical):

```
goal = [[1,2,3],[4,5,6],[7,8,0]]

def goal_position(v):
    for i in range(3):
        for j in range(3):
            if goal[i][j] == v: return (i, j)

def manhattan(state):
    d = 0
    for i in range(3):
        for j in range(3):
            v = state[i][j]
            if v != 0:                      # skip blank!
                gi, gj = goal_position(v)
                d += abs(i-gi) + abs(j-gj)
    return d
```

2b. BFS (P1 Q2): flat tuple version

```
def bfs(initial, goal):
    queue = deque([(initial, [], [initial])])   # state, moves, states
    visited = {initial}
    while queue:
        cur, path, state_path = queue.popleft()
        if cur == goal:
            return path, state_path
        for nb, move in get_neighbors(cur):
            if nb not in visited:
                visited.add(nb)
                queue.append((nb, path+[move], state_path+[nb]))
    return None, None
```

2c. Greedy Best-First (P3): 2D version

```
import heapq

def greedy_best_first(start):
    pq = [(manhattan(start), start, [start])]
    visited = set()
    while pq:
        h, cur, path = heapq.heappop(pq)
        if cur == goal: return path
        t = state_to_tuple(cur)
        if t in visited: continue
        visited.add(t)
        for nb in get_neighbors(cur):
            if state_to_tuple(nb) not in visited:
                heapq.heappush(pq, (manhattan(nb), nb, path+[nb]))
    return None
```

2d. A* for 8-puzzle (P4 Q1): only 3 changes from Greedy

1. Push tuple is `(f, g, state, path)` instead of `(h, state, path)`.
2. `visited` becomes a **dict** `{state_tuple: g}` and you skip if `visited[t] <= g`.
3. `new_g = g + 1`, `new_f = new_g + manhattan(nb)`.

```
def a_star(start):
    pq = [(manhattan(start), 0, start, [start])]
    visited = {}
    while pq:
        f, g, cur, path = heapq.heappop(pq)
        if cur == goal: return path
        t = state_to_tuple(cur)
        if t in visited and visited[t] <= g: continue
        visited[t] = g
        for nb in get_neighbors(cur):
            new_g = g + 1
            heapq.heappush(pq, (new_g + manhattan(nb), new_g, nb, path+[nb]))
    return None
```

2e. Graph search: UCS (P2 Q2) vs A* (P4 Q2)

```
# UCS: graph = {'Mumbai': [('Panvel', 40), ...], ...}
def uniform_cost_search(graph, start, goal):
    pq = [(0, start, [start])]
    visited = set()
    while pq:
        cost, city, path = heapq.heappop(pq)
        if city == goal: return cost, path
        if city not in visited:
            visited.add(city)
            for nb, dist in graph[city]:
                if nb not in visited:
                    heapq.heappush(pq, (cost+dist, nb, path+[nb]))
    return None
```

```
# A*: same graph PLUS heuristic dict (straight-line distance to goal)
heuristic = {'Mumbai':120, 'Thane':110, 'Panvel':95, 'Lonavala':55, 'Pune':0}

def a_star(start, goal):
    pq = [(heuristic[start], 0, start, [start])]
    visited = {}
    while pq:
        f, g, city, path = heapq.heappop(pq)
        if city == goal: return path, g
        if city in visited and visited[city] <= g: continue
        visited[city] = g
        for nb, dist in graph[city]:
            new_g = g + dist
            heapq.heappush(pq, (new_g + heuristic[nb], new_g, nb, path+[nb]))
    return None, None
```

*UCS vs A differences:** UCS has no heuristic and tuple is `(cost, city, path)`. A* adds a `heuristic` dict, tuple is `(f, g, city, path)`, and `visited` is a dict of best g.

For Arad → Bucharest (Romania map), only the `graph` dict and `start/goal` change. For A* on it you also need an SLD heuristic table (Arad 366, Sibiu 253, Rimnicu Vilcea 193, Pitesti 100, Fagaras 176, Bucharest 0, etc.).

2f. DFS: Water Jug (P2 Q1)

State = `(x, y)` = water in jug 1 and jug 2. **Six moves** = fill 1, fill 2, empty 1, empty 2, pour 1→2, pour 2→1.

```
def dfs(state, visited, path, cap1, cap2, target):
    if state[0] == target or state[1] == target:
        path.append(state); return True
    if state in visited: return False
    visited.add(state)
    path.append(state)
    x, y = state
    next_states = [
        (cap1, y), (x, cap2), (0, y), (x, 0),
        (x - min(x, cap2-y), y + min(x, cap2-y)),   # pour 1 -> 2
        (x + min(y, cap1-x), y - min(y, cap1-x)),   # pour 2 -> 1
    ]
    for ns in next_states:
        if dfs(ns, visited, path, cap1, cap2, target): return True
```

```
    path.pop()          # backtrack
    return False

cap1 = int(input("Jug 1: ")); cap2 = int(input("Jug 2: ")); target =
int(input("Target: "))
visited, path = set(), []
if dfs((0,0), visited, path, cap1, cap2, target):
    for i, s in enumerate(path): print("Step", i, ":", s)
else:
    print("No Solution Exists")
```

To "vary capacity" you just type different inputs. A solution exists only if target ⩽ max(cap1, cap2) and target is a multiple of gcd(cap1, cap2).

---

3. T3: Bayes' rule (P5 Q1)

Formula: **P(D|+) = P(+|D)·P(D) / [P(+|D)·P(D) + P(+|¬D)·P(¬D)]**

```
p_d = 0.01
p_pos_d = 0.99
p_pos_nd = 0.05
p_nd = 1 - p_d

p_d_pos = (p_pos_d * p_d) / (p_pos_d * p_d + p_pos_nd * p_nd)

print("P(Disease | Positive) =", p_d_pos)
print("Percentage:", p_d_pos * 100, "%")
```

Answer ≈ **0.1667 (16.67%)**.

Syntax warning: in your journal the expression is split across lines without brackets, which gives a `SyntaxError`. Put the whole right-hand side in one pair of `( )` or on one line.

---

4. T4: The ML workflow (P5 Q2, P6, P7, P8)

Every ML practical is these 5 lines:

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
model = SomeModel()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print(accuracy_score(y_test, y_pred))        # classification
# print(mean_squared_error(y_test, y_pred), r2_score(y_test, y_pred))  #
regression
```

Rule: classification → `accuracy_score`; regression → `mean_squared_error` + `r2_score`.

What changes in each practical

Practical	Dataset (loader)	Model + import	Extra step	Metric
P5 Q2	Built with `pd.DataFrame` (Temp, Humidity, WindSpeed → Play)	`from sklearn.naive_bayes import GaussianNB`	`classification_report`, predict a new day	accuracy
P6 Q1	`load_iris()` + `pd.DataFrame`	`DecisionTreeClassifier`	`StandardScaler` before split; print **train and test** accuracy	accuracy
P6 Q2	`pd.read_csv("Student_Performance.csv")`	`LinearRegression` (`sklearn.linear_model`)	`X = df[["Hours Studied"]]` (double brackets); print `coef_`, `intercept_`	MSE, R^2
P6 Q3	`fetch_openml("mnist_784", version=1, as_frame=False)`	`SGDClassifier` (`sklearn.linear_model`)	`y = (y == "5")` makes it binary	train + test accuracy
P7 Q1	`load_iris()`	`DecisionTreeClassifier`	`StratifiedKFold(n_splits=5, shuffle=True, random_state=42)` loop	accuracy per fold + average
P7 Q2	`load_wine()`	`DecisionTreeClassifier(max_depth=4)`	none	accuracy
P7 Q3	`load_diabetes()`	`DecisionTreeRegressor(max_depth=4)`	predict a new 10-feature row	MSE, R^2
P8 Q1	`load_wine()`	`KNeighborsClassifier(n_neighbors=5)`	`StandardScaler`	accuracy
P8 Q2	`load_wine()`	`OneVsOneClassifier(LogisticRegression(max_iter=1000))`	`StandardScaler`	accuracy

Imports cheat list

```
import pandas as pd
from sklearn.datasets import load_iris, load_wine, load_diabetes,
fetch_openml
from sklearn.model_selection import train_test_split, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report,
mean_squared_error, r2_score
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor
from sklearn.linear_model import LinearRegression, LogisticRegression,
SGDClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.multiclass import OneVsOneClassifier
```

Scaling: the one trick to remember

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)   # fit ONLY on train
X_test  = scaler.transform(X_test)        # just transform test
```

(P6 Q1 scales the full `X` before splitting, which is simpler but leaks test statistics. In P8 the correct order is used. Prefer the P8 way and say "to avoid data leakage".)

Quick differences between the models

Model	Type	Key idea	Needs scaling?
Naive Bayes	classification	Bayes' rule + assumes features independent	No
Decision Tree	classif./regression	splits on features (Gini/entropy; MSE for regression)	No
Linear Regression	regression	best-fit line y = mx + c	Not required
Logistic Regression	classification	sigmoid gives probability; OvO/OvR for multiclass	Yes

Model	Type	Key idea	Needs scaling?
kNN	classification	majority vote of k nearest points	**Yes** (distance based)
SGDClassifier	classification	linear model trained by stochastic gradient descent	Yes

P7 Q1 k-fold loop

```
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
accuracies = []
for fold, (tr, te) in enumerate(skf.split(X, y), 1):
    model.fit(X[tr], y[tr])
    acc = accuracy_score(y[te], model.predict(X[te]))
    accuracies.append(acc)
    print("Fold", fold, "Accuracy:", acc)
print("Average:", sum(accuracies)/len(accuracies))
```

P5 Q2 and P7 Q3 extras

```
# predict a new sample (P5 Q2): keep column names
new_day = pd.DataFrame([[26, 68, 10]], columns=
["Temperature","Humidity","WindSpeed"])
print(model.predict(new_day)[0])

# predict new sample (P7 Q3): list of lists, 10 values
print(model.predict([[0.05,0.05,0.06,0.02,-0.01,0.01,0.02,0.01,0.03,0.02]])
[0])
```

---

5. T5: Bayesian Network with pgmpy (P5 Q3)

Structure to memorise: Burglary → Alarm ← Earthquake; Alarm → JohnCalls; Alarm → MaryCalls.

6 steps: (1) model with edges → (2) CPD for root nodes → (3) CPD for Alarm (2 parents, 4 columns) → (4) CPD for John, Mary (1 parent, 2 columns) → (5) `add_cpds` + `check_model` → (6) `VariableElimination` queries.

```
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination

model = DiscreteBayesianNetwork([
    ('Burglary','Alarm'), ('Earthquake','Alarm'),
    ('Alarm','JohnCalls'), ('Alarm','MaryCalls')
])

cpd_b = TabularCPD('Burglary', 2, [[0.999],[0.001]],
                   state_names={'Burglary':['False','True']})
cpd_e = TabularCPD('Earthquake', 2, [[0.998],[0.002]],
                   state_names={'Earthquake':['False','True']})

cpd_a = TabularCPD('Alarm', 2,
    [[0.999, 0.06, 0.71, 0.05],      # Alarm=False
     [0.001, 0.94, 0.29, 0.95]],     # Alarm=True
    evidence=['Burglary','Earthquake'], evidence_card=[2,2],
    state_names={'Alarm':['False','True'],
                 'Burglary':['False','True'],
                 'Earthquake':['False','True']})

cpd_j = TabularCPD('JohnCalls', 2, [[0.95,0.10],[0.05,0.90]],
    evidence=['Alarm'], evidence_card=[2],
    state_names={'JohnCalls':['False','True'], 'Alarm':['False','True']})

cpd_m = TabularCPD('MaryCalls', 2, [[0.99,0.30],[0.01,0.70]],
    evidence=['Alarm'], evidence_card=[2],
    state_names={'MaryCalls':['False','True'], 'Alarm':['False','True']})

model.add_cpds(cpd_b, cpd_e, cpd_a, cpd_j, cpd_m)
print(model.check_model())

inf = VariableElimination(model)
print(inf.query(['Burglary']))
print(inf.query(['Burglary'], evidence={'Alarm': 'True'}))     # note the
quotes
print(inf.query(['Burglary'], evidence={'JohnCalls':'True',
'MaryCalls':'True'}))
```

Column order rule for the Alarm table: columns go (B,E) = (F,F), (F,T), (T,F), (T,T). Each column must sum to 1.

Bug in your journal: `evidence={'Alarm': True}` (Python boolean) will fail or return an error because `state_names` are the **strings** `'False'` / `'True'`. Use `{'Alarm': 'True'}`.

---

6. Known problems in the journal code (fix before the exam)

Where	Problem	Fix
P5 Q1	Expression spans lines without brackets, `SyntaxError`	Wrap in `( )` or write on one line
P5 Q3	`evidence={'Alarm': True}` uses bool	Use `'True'` (string)
P5 Q2	`new_day = ...prediction = model.predict(...)` joined on one line	Put on two separate lines
P3 / P4	`heapq` compares the state lists when two f-values tie; works for lists, but `goal` must be a list of lists to compare with `==`	Keep `goal` as list of lists
P3 / P4	Start state must be solvable (even number of inversions) else search runs forever	Use the given sample states
P6 Q2	Needs `Student_Performance.csv` in the same folder	Put CSV in working directory
P6 Q3	Needs internet for `fetch_openml`; slow on 70k images	Warn the examiner it takes time
P5 Q3	`DiscreteBayesianNetwork` exists only in new pgmpy; older versions use `BayesianNetwork`	Use whichever your lab PC accepts

---

7. Expected outputs (so you know your run is correct)

- **P1 Q1:** `('C','Dirty')` → `No Action`.
- **P1 Q2 / P3 / P4 Q1:** start `1 2 3 / 4 0 6 / 7 5 8` → goal `1 2 3 / 4 5 6 / 7 8 0` takes **2 moves** (Right, Down for the blank, i.e. blank moves right then down).
- **P2 Q2:** Mumbai → Lonavala → Pune, cost **150**.
- **P2 Q2a:** Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bucharest, cost **418**.

- **P4 Q2:** Mumbai → Panvel → Lonavala → Pune, cost **185 km** (your A* graph has no direct Mumbai–Lonavala edge).
- **P5 Q1:** about **16.67%**.
- **P6 Q1 (Iris tree):** train accuracy **1.0**, test about **1.0**.
- **P7 Q1:** 5 fold accuracies, average about 0.95 to 0.96.

---

8. Viva and theory one-liners

- **Rational agent:** picks the action that maximises expected performance measure given percepts so far.
- **Table-driven agent drawback:** table size explodes with the percept sequence length.
- **BFS:** complete, optimal for unit costs, memory heavy O(b^d). **DFS:** low memory, not optimal, may loop without a visited set.
- **UCS:** expands the lowest g(n); equals BFS when all step costs are equal.
- **Greedy:** uses only h(n); fast but neither optimal nor complete in general.
- *A:** f = g + h; optimal when h is **admissible** (never overestimates). Manhattan distance and straight-line distance are both admissible.
- **Manhattan distance:** sum of |row difference| + |column difference| for each tile, blank excluded.
- **Bayes:** posterior = likelihood × prior / evidence.
- **Overfitting sign:** train accuracy ≫ test accuracy (e.g. unpruned decision tree: train 1.0).
- **Why k-fold:** every sample is used for both training and testing, giving a more reliable estimate than a single split.
- **Stratified:** each fold keeps the same class proportions.
- **MSE:** average squared error (lower is better). **R^2:** fraction of variance explained (closer to 1 is better).
- **One-vs-One:** trains a binary classifier for each pair of classes (3 classes → 3 classifiers).
- **Binary classification (MNIST):** label 5 vs not-5; accuracy can look high (~90%) just by predicting "not 5" because classes are imbalanced.

---

9. Exam strategy (15 marks, 1 or 2 questions)

1. **Write the algorithm steps first** (2 to 3 marks usually). You have them in your journal in short form.
2. Type the **imports and data** first, then the function, then the **driver code** that prints the result.
3. If you blank on code: write the template (frontier, visited, loop, neighbors) and fill in. Partial code earns marks.
4. Always print results clearly (`print("Accuracy:", accuracy)`), since the output is part of the marks.
5. Practise by **re-typing only the changed lines** between related programs: BFS → Greedy → A*, Greedy → A* graph, DecisionTree → kNN → Logistic (just swap the model line).